

1.

ANS 1. A customer's PAN number, always exactly 10 characters.

Data Type: CHAR(10) Reason: PAN is always exactly 10 characters and is not used for calculations.

2. The interest amount credited on a fixed deposit at maturity.

Data Type: DECIMAL(15,2) Reason: Stores monetary values accurately with 2 decimal places.

3. The number of days a deposit runs, maximum 3,650.

Data Type: SMALLINT Reason: Stores whole numbers, and 3,650 fits easily within its range.

4. The exact moment a UPI payment was authorised.

Data Type: TIMESTAMP Reason: Stores both the date and exact time.

5. The date a loan instalment falls due. Data Type: DATE Reason: Only the calendar date is required.

6. A free-text branch note about a customer complaint.

Data Type: TEXT Reason: Suitable for variable-length written notes.

7. A running counter of every transaction the bank has ever posted.

Data Type: BIGINT Reason: Can store a very large whole-number transaction counter.

8. Whether a deposit auto-renews.

Data Type: BOOLEAN Reason: Stores a simple true/false value.

2.

A developer proposes `interest\_amount FLOAT` because "it handles decimals". Write the two-line SQL demonstration you would show them, and state in one sentence what the business impact is.

ANS: `sql SELECT 0.1 + 0.2 AS float_result;`

`SELECT CAST(0.1 AS DECIMAL(10,2)) + CAST(0.2 AS DECIMAL(10,2)) AS decimal_result;`

Business Impact: `FLOAT` can introduce rounding errors in monetary calculations, which may cause incorrect interest amounts and financial discrepancies.

3.

ANS:

Constraint	What it guarantees	ICICI Bank example
NOT NULL	Column must always contain a value	customer.pan_number — every customer must have a PAN on file
UNIQUE	No two rows may share the same value (NULLs generally exempted)	fixed_deposit.receipt_number — receipt numbers must never repeat
PRIMARY KEY	Uniquely identifies each row; implies NOT NULL + UNIQUE	fixed_deposit.deposit_id
FOREIGN KEY	A column's value must exist in the referenced table's key column	fixed_deposit.customer_id must reference an existing customer.customer_id

CHECK	Restricts values to a defined rule/range	fixed_deposit.status IN ('ACTIVE', 'MATURED', 'BROKEN', 'CLOSED')
DEFAULT	Supplies a value automatically when none is given on insert	recurring_deposit.status DEFAULT 'ACTIVE'

4.State two differences between a **PRIMARY KEY** and a **UNIQUE** constraint, and explain why **deposit_id** was chosen as the primary key rather than **receipt_number**.

ANS: A **PRIMARY KEY** uniquely identifies each row and cannot contain NULL; a **UNIQUE** constraint prevents duplicate values but can allow NULL depending on the database.

A table can have only one **PRIMARY KEY**, but it can have multiple **UNIQUE** constraints.

Why **deposit_id** instead of **receipt_number**?

deposit_id is chosen as the primary key because it is intended to be the unique internal identifier of the deposit, while **receipt_number** is a business/reference number.

5.

ANS:

Dimension	DELETE	TRUNCATE	DROP
What is removed	Selected rows (or all rows)	All rows, table structure stays	The entire table object (structure + data)
WHERE allowed	Yes	No	N/A
Rollback-able	Yes (logged row-by-row)	Usually yes in PostgreSQL; not in MySQL/InnoDB by default	Yes in PostgreSQL within a transaction; not in most other RDBMSs
Triggers fire	Yes	No (row-level triggers don't fire)	No
Speed on large table	Slow — row-by-row logging	Fast — deallocates pages directly	Fast — removes the object entirely

6.ICICI Bank must keep seven years of transaction history for the regulator. A developer writes a monthly job that runs DELETE FROM account_txn WHERE txn_date < CURRENT_DATE - 90. State what is wrong and what the bank should do instead.

ANS: Problem:

The query deletes transactions older than only 90 days, which violates the requirement to retain seven years of transaction history.

What the bank should do:

The bank should retain at least seven years of transaction data and use a controlled archival/purge policy only for data older than the required retention period.

7.

ANS:

- 1) NULL
- 2) NULL
- 3) NULL
- 4) FALLBACK
- 5) AB

8.

ANS: AND has higher precedence than OR.

Therefore, Query 1 is interpreted as:

WHERE branch_id = 1

OR (branch_id = 2 AND principal_amount > 500000);

Query 2 explicitly applies the amount condition to both branches.

9.

ANS: WHERE txn_date >= DATE '2026-01-01'

AND txn_date < DATE '2026-04-01';

10.

ANS: SQL evaluates WHERE before SELECT, so the alias does not exist when WHERE is evaluated. ORDER BY is evaluated after SELECT, so it can use the SELECT alias.

11.

ANS:

```
CREATE TABLE recurring_deposit (  
rd_id BIGINT CONSTRAINT pk_recurring_deposit PRIMARY KEY,
```

```
customer_id BIGINT  
CONSTRAINT fk_rd_customer  
REFERENCES customer(customer_id),
```

```
branch_id BIGINT  
CONSTRAINT fk_rd_branch  
REFERENCES branch(branch_id),
```

```
rd_number VARCHAR(20)  
CONSTRAINT uq_rd_number UNIQUE,
```

```
monthly_amount DECIMAL(15,2)  
CONSTRAINT chk_rd_monthly_amount CHECK (monthly_amount >=  
500),
```

interest_rate DECIMAL(5,2)
CONSTRAINT chk_rd_interest_rate
CHECK (interest_rate > 0 AND interest_rate <= 12),

tenure_months SMALLINT
CONSTRAINT chk_rd_tenure
CHECK (tenure_months BETWEEN 6 AND 120),

start_date DATE
CONSTRAINT nn_rd_start_date NOT NULL
DEFAULT CURRENT_DATE,

maturity_date DATE
CONSTRAINT nn_rd_maturity_date NOT NULL,

installments_paid INTEGER
CONSTRAINT chk_rd_installments
CHECK (installments_paid >= 0)
DEFAULT 0,

status VARCHAR(10)
CONSTRAINT chk_rd_status
CHECK (status IN ('ACTIVE', 'MATURED', 'DEFAULTED', 'CLOSED'))
DEFAULT 'ACTIVE',

CONSTRAINT chk_rd_maturity
CHECK (maturity_date > start_date)

);

12.

ANS 1) An RD with monthly_amount of 100.

Query	Query History
1	INSERT INTO recurring_deposit
2	(rd_id, customer_id, branch_id, rd_number, monthly_amount,
3	interest_rate, tenure_months, start_date, maturity_date)
4	VALUES
5	(9001, 1001, 1, 'RD_INVALID1', 100, 7.10, 24,
6	DATE '2026-03-01', DATE '2028-03-01');

2) An RD for customer_id 5555.

Query	Query History
1	INSERT INTO recurring_deposit
2	(rd_id, customer_id, branch_id, rd_number, monthly_amount,
3	interest_rate, tenure_months, start_date, maturity_date)
4	VALUES
5	(9002, 5555, 1, 'RD_INVALID2', 5000, 7.10, 24,
6	DATE '2026-03-01', DATE '2028-03-01');

3) An RD whose maturity_date is before its start_date.

Query	Query History
1	INSERT INTO recurring_deposit
2	(rd_id, customer_id, branch_id, rd_number, monthly_amount,
3	interest_rate, tenure_months, start_date, maturity_date)
4	VALUES
5	(9003, 1001, 1, 'RD_INVALID3', 5000, 7.10, 24,
6	DATE '2026-03-01', DATE '2025-03-01');

4) Two RDs with the same rd_number.

```
1  INSERT INTO recurring_deposit
2  (rd_id, customer_id, branch_id, rd_number, monthly_amount,
3   interest_rate, tenure_months, start_date, maturity_date)
4  VALUES
5  (9004, 1001, 1, 'RD2026000001', 5000, 7.10, 24,
6   DATE '2026-03-01', DATE '2028-03-01');|
```

5) An RD with status of PAUSED.

Query	Query History
1 INSERT INTO recurring_deposit	
2 (rd_id, customer_id, branch_id, rd_number, monthly_amount,	
3 interest_rate, tenure_months, start_date, maturity_date, status)	
4 VALUES	
5 (9005, 1001, 1, 'RD_INVALID5', 5000, 7.10, 24,	
6 DATE '2026-03-01', DATE '2028-03-01', 'PAUSED');	

6) An RD with installments_paid of -3.

```
1  INSERT INTO recurring_deposit
2  (rd_id, customer_id, branch_id, rd_number, monthly_amount,
3   interest_rate, tenure_months, start_date, maturity_date,
4   installments_paid)
5  VALUES
6  (9006, 1001, 1, 'RD_INVALID6', 5000, 7.10, 24,
7   DATE '2026-03-01', DATE '2028-03-01', -3);|
```

13.

Q13. Insert these three recurring deposits in a single statement.

rd_id	customer_id	branch_id	rd_number	monthly	rate	tenure	start	maturity
8001	1001	1	RD2026000001	5000	7.10	24	2026-03-01	2028-03-01
8002	1005	5	RD2026000002	2500	6.85	12	2026-03-05	2027-03-05
8003	1012	4	RD2026000003	10000	7.35	60	2026-03-10	2031-03-10

```
Query  Query History
1  INSERT INTO recurring_deposit
2  (rd_id, customer_id, branch_id, rd_number, monthly_amount,
3   interest_rate, tenure_months, start_date, maturity_date)
4  VALUES
5  (8001, 1001, 1, 'RD2026000001', 5000, 7.10, 24,
6   DATE '2026-03-01', DATE '2028-03-01'),
7
8  (8002, 1005, 5, 'RD2026000002', 2500, 6.85, 12,
9   DATE '2026-03-05', DATE '2027-03-05'),
10
11 (8003, 1012, 4, 'RD2026000003', 10000, 7.35, 60,
12  DATE '2026-03-10', DATE '2031-03-10');
```

14. The compliance team asks for the RD table to record a nominee. Add a nullable `nominee_name` column of up to 60 characters, then set the nominee for RD 8001 to 'Meera Reddy'.

ANS:

```
Query  Query History
1  ALTER TABLE recurring_deposit
2  ADD COLUMN nominee_name VARCHAR(60);
3
4  UPDATE recurring_deposit
5  SET nominee_name = 'Meera Reddy'
6  WHERE rd_id = 8001;
```

15. Product decides the minimum monthly instalment rises to 1,000. Replace the existing check constraint. Before you do, write the query that tells you whether any existing row would violate the new rule.

Query	Query History
1	SELECT *
2	FROM recurring_deposit
3	WHERE monthly_amount < 1000;
4	
5	ALTER TABLE recurring_deposit
6	DROP CONSTRAINT chk_rd_monthly_amount;
7	
8	ALTER TABLE recurring_deposit
9	ADD CONSTRAINT chk_rd_monthly_amount
10	CHECK (monthly_amount >= 1000);

16. Try to add a mandatory `scheme_code` `VARCHAR(8)` `NOT NULL` column to the populated `recurring_deposit` table. Record the error, then make it succeed.

Query	Query History
1	<code>ALTER TABLE recurring_deposit</code>
2	<code>ADD COLUMN scheme_code VARCHAR(8) NOT NULL;</code>
3	
4	<code>ALTER TABLE recurring_deposit</code>
5	<code>ADD COLUMN scheme_code VARCHAR(8);</code>
6	
7	<code>UPDATE recurring_deposit</code>
8	<code>SET scheme_code = 'RD001';</code>
9	
10	<code>ALTER TABLE recurring_deposit</code>
11	<code>ALTER COLUMN scheme_code SET NOT NULL;</code>
12	

17. List every fixed deposit of 500,000 or more, showing receipt number, principal, rate and maturity date, largest first.

Query	Query History
1	<code>SELECT receipt_number,</code>
2	<code>principal_amount,</code>
3	<code>interest_rate,</code>
4	<code>maturity_date</code>
5	<code>FROM fixed_deposit</code>
6	<code>WHERE principal_amount >= 500000</code>
7	<code>ORDER BY principal_amount DESC;</code>

18. List fixed deposits booked between 1 and 10 March 2026 inclusive, sorted by booked date then principal descending.

Query	Query History
1	SELECT *
2	FROM fixed_deposit
3	WHERE booked_date >= DATE '2026-03-01'
4	AND booked_date < DATE '2026-03-11'
5	ORDER BY booked_date,
6	principal_amount DESC ;

19. Show every fixed deposit with no nominee recorded. Then show every fixed deposit whose nominee is not 'Priya Verma', including those with no nominee at all. Explain in one sentence why the second query needs more than a simple $\diamond$.

Query	Query History
1	SELECT *
2	FROM fixed_deposit
3	WHERE nominee_name IS NULL ;
4	
5	SELECT *
6	FROM fixed_deposit
7	WHERE nominee_name <> 'Priya Verma'
8	OR nominee_name IS NULL ;

20. For each fixed deposit show the receipt number, principal, and a computed column `annual_interest` equal to principal times rate divided by 100, rounded to 2 decimals. Sort by annual interest descending and show the top five.

Query	Query History
1	<code>SELECT receipt_number,</code>
2	<code>principal_amount,</code>
3	<code>ROUND(principal_amount * interest_rate / 100, 2)</code>
4	<code>AS annual_interest</code>
5	<code>FROM fixed_deposit</code>
6	<code>ORDER BY annual_interest DESC</code>
7	<code>LIMIT 5;</code>

21. Produce a list for the branch, showing for every fixed deposit: receipt number, customer full name as one column, principal, and a **segment** column that reads HIGH VALUE at 1,000,000 or above, STANDARD at 100,000 or above, and SMALL otherwise. Sort high value first.

Query	Query History
1	SELECT
2	fd.receipt_number,
3	CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
4	fd.principal_amount,
5	CASE
6	WHEN fd.principal_amount >= 1000000 THEN 'HIGH VALUE'
7	WHEN fd.principal_amount >= 100000 THEN 'STANDARD'
8	ELSE 'SMALL'
9	END AS segment
10	FROM fixed_deposit fd
11	JOIN customer c
12	ON fd.customer_id = c.customer_id
13	ORDER BY
14	CASE
15	WHEN fd.principal_amount >= 1000000 THEN 1
16	WHEN fd.principal_amount >= 100000 THEN 2
17	ELSE 3
18	END;

22. Find all transactions in **account_txn** whose narration came through the UPI channel and whose amount exceeds 50,000. Show transaction id, account id, amount and narration, largest first, limited to ten rows. Write the **LIMIT** clause three ways: PostgreSQL, SQL Server and Oracle.

Query	Query History
1	SELECT txn_id, account_id, amount, narration
2	FROM account_txn
3	WHERE narration ILIKE '%UPI%'
4	AND amount > 50000
5	ORDER BY amount DESC
6	LIMIT 10;
7	
8	SELECT TOP 10 txn_id, account_id, amount, narration
9	FROM account_txn
10	WHERE narration LIKE '%UPI%'
11	AND amount > 50000
12	ORDER BY amount DESC ;
13	
14	SELECT txn_id, account_id, amount, narration
15	FROM account_txn
16	WHERE UPPER (narration) LIKE '%UPI%'
17	AND amount > 50000
18	ORDER BY amount DESC
19	FETCH FIRST 10 ROWS ONLY ;

23. Deposit 7009 is being broken early by the customer. Write the **SELECT** that identifies it, then the **UPDATE** that sets status to BROKEN and auto-renew to N, then the **SELECT** that verifies the change. Roll the whole thing back at the end.

Query	Query History
-------	---------------

1	1. Identify the deposit:
2	BEGIN;
3	
4	SELECT *
5	FROM fixed_deposit
6	WHERE deposit_id = 7009;
7	2. Update the deposit:
8	UPDATE fixed_deposit
9	SET status = 'BROKEN',
10	auto_renew = 'N'
11	WHERE deposit_id = 7009;
12	3. Verify the change:
13	SELECT *
14	FROM fixed_deposit
15	WHERE deposit_id = 7009;
16	4. Roll back the changes:
17	ROLLBACK;

24. The nightly maturity job marks as MATURED every ACTIVE deposit whose maturity date is on or before 30 June 2026. Write the **SELECT** first, note the count, run the **UPDATE**, and confirm the counts match. Roll back afterwards.

Query	Query History
1	1. Find eligible deposits:
2	BEGIN;
3	
4	SELECT *
5	FROM fixed_deposit
6	WHERE status = 'ACTIVE'
7	AND maturity_date <= DATE '2026-06-30';
8	2. Check the count:
9	SELECT COUNT(*)
10	FROM fixed_deposit
11	WHERE status = 'ACTIVE'
12	AND maturity_date <= DATE '2026-06-30';
13	3. Update the deposits:
14	UPDATE fixed_deposit
15	SET status = 'MATURED'
16	WHERE status = 'ACTIVE'
17	AND maturity_date <= DATE '2026-06-30';
18	4. Confirm the updated count:
19	SELECT COUNT(*)
20	FROM fixed_deposit
21	WHERE status = 'MATURED'
22	AND maturity_date <= DATE '2026-06-30';
23	5. Roll back:
24	ROLLBACK;

Data Output	Messages	Notifications
-------------	----------	---------------

25. RD 8002 misses three instalments and is marked defaulted. Update `status` and leave `installments_paid` unchanged. Then, instead of deleting the row, close it the way a bank would, and explain in one line why.

Query	Query History
1	1. Mark RD 8002 as DEFAULTED:
2	UPDATE recurring_deposit
3	SET status = 'DEFAULTED'
4	WHERE rd_id = 8002;
5	2. Verify the status and installments:
6	SELECT rd_id, status, installments_paid
7	FROM recurring_deposit
8	WHERE rd_id = 8002;
9	3. Close the RD instead of deleting it:
10	UPDATE recurring_deposit
11	SET status = 'CLOSED'
12	WHERE rd_id = 8002;
13	4. Verify the final status:
14	SELECT rd_id, status, installments_paid
15	FROM recurring_deposit
16	WHERE rd_id = 8002;

26. A vendor's data feed for the Smart Saver product supplies `principal_amount` as a text field, sometimes as "250000", sometimes as "2,50,000.00", and occasionally as "250000 INR". The vendor suggests you store the column as `VARCHAR(30)` and "parse it when needed". Write your response. Include what you would insist on, and where in the pipeline you would put the cleaning.

ANS:Store principal_amount as DECIMAL(15,2), not VARCHAR. Clean and convert the data in the ETL/data-ingestion pipeline before inserting it into the database.

27.Two engineers disagree. One wants ON DELETE CASCADE on fixed_deposit.customer_id, arguing that closing a customer should tidy up their deposits automatically. The other wants RESTRICT. Take a position and defend it in five sentences, referring to what a bank actually does when a customer leaves.

ANS:Choose RESTRICT. A customer's departure should not automatically delete deposit records because banking records must be retained for history, audit and regulatory purposes.

28.ICICI Bank's account_txn table will hold 40 million rows. A new requirement says every transaction must record which employee authorised it, where applicable. Design the change: state the column, its type, its nullability, whether it gets a foreign key, and what you would do about the 40 million existing rows.

ANS:Column: authorised_by_employee_id

Type: Same type as employee primary key

Nullable: NULL where not applicable

Foreign Key: Yes

Existing 40M rows: Keep NULL initially and populate where reliable historical data exists.

29.The status column on fixed_deposit is VARCHAR(12) with a CHECK restricting it to four values. An architect proposes replacing it with a separate deposit_status lookup table and a foreign key. List two advantages of each approach and say which you would choose for this table and why.

ANS:For this table, choose a lookup table + FOREIGN KEY if statuses may grow or need extra information; otherwise, CHECK is simpler.

30:A junior colleague has just run UPDATE account SET status = 'DORMANT'; on the production database, with no WHERE clause and autocommit on. They

come to you two minutes later. Write what you would do, in order, and then write the two changes you would make to the team's working practices so it does not happen again. Do not blame the individual.

ANS: Stop further changes immediately.

Check whether recovery/rollback is possible.

Find the number of affected rows.

Use backups/audit logs to identify original values.

Restore the correct values.

Verify the corrected rows.

Record the incident.

Add production change-review controls.

Test with SELECT before UPDATE.

Use transactions and safeguards for production changes.